

MuJoCo MPC 汽车仪表盘 - 作业报告

一、项目概述

1.1 作业背景

本次大作业是 C++ 课程的重要组成部分，要求基于 Google DeepMind 开源的 MuJoCo MPC (Model Predictive Control) 框架进行二次开发。MuJoCo 是一个高性能的物理引擎，广泛应用于机器人和自动驾驶领域的仿真研究。通过本项目，旨在深入理解物理引擎的工作原理，掌握大型 C++ 开源项目的代码结构，并提升在仿真环境中进行数据可视化和交互界面开发的能力。

1.2 实现目标

本项目的主要目标是将车辆的物理状态实时可视化，构建一个功能完备的“仿真汽车仪表盘”。具体实现的子目标包括：

- 基础数据可视化**：实时获取并显示车辆的速度、转速、油量等核心数据。
- 高级逻辑模拟**：实现自动变速箱（自动换挡）逻辑和驾驶模式（ECO/SPORT）切换逻辑。
- 视觉增强**：在界面右下角集成“画中画”小地图，提供上帝视角的车辆追踪功能。
- 安全交互**：基于接触动力学实现碰撞检测系统，并提供全屏视觉警告。

1.3 开发环境

- 操作系统**: Ubuntu 22.04 LTS (WSL2)
- 编译器**: gcc 11.3.0 (build-essential)
- 构建工具**: CMake 3.22.1
- 核心库**: MuJoCo (项目内置), 标准 C++ 库
- 辅助库**: lodepng (用于截图保存)

二、技术方案

2.1 系统架构

本项目采用了“嵌入式集成”的架构方案。为了确保渲染的高效性与数据的实时同步，核心逻辑直接植入于 `mjpc` 的主渲染循环中。

- 模块划分**：
- 数据提取层**：直接访问 `mjData` 结构体，读取 `qvel` (速度)、`qacc` (加速度)、`contact` (接触点) 等物理数据。

- **逻辑处理层**：在渲染前进行数据计算，包括速度单位转换、档位状态机判断、驾驶模式判定及油耗迭代计算。
- **渲染表现层**：利用 MuJoCo 原生的 `mjr_` 系列函数（UI 覆盖层）和 `mjv_` 系列函数（场景摄像机）进行最终的画面输出。

2.2 数据流程

物理步进 (Physics Step)：MuJoCo 引擎更新 `mjData`，计算当前帧的物理状态。 **状态获取**：在 `Simulate::Render()` 函数中，每一帧读取车辆的线性速度向量和加速度向量。 **逻辑运算**：

- 速度合成：
 - 档位映射：根据 的大小映射到 P/R/N/D1-D5。
 - 碰撞判定：遍历 `mjData->contact` 数组，检测车身几何体 ID 是否参与碰撞。
4. **UI 绘制**：将计算后的字符串和几何图形参数传递给 `mjr_overlay` 和 `mjr_rectangle` 进行绘制。

2.3 渲染方案

为了解决作业初期遇到的 OpenGL 链接库兼容性问题，本项目最终选用了 **MuJoCo Native UI** 方案：

- **文字与数据**：使用 `mjr_overlay` 函数。该函数支持在屏幕指定区域（如左下角 `mjGRID_BOTTOMLEFT`）绘制格式化文本，无需手动管理字体纹理。
- **图形元素**：使用 `mjr_rectangle` 函数绘制油量条、小地图边框及红色警告框。该函数支持 RGBA 颜色混合，实现了半透明的 HUD 效果。
- **画中画 (PIP)**：利用 `mjvCamera` 创建第二个摄像机实例，设置为 `mjCAMERA_TRACKING` 模式并垂直俯视，通过 `mjr_render` 将其视口渲染到屏幕右下角的特定矩形区域。

三、实现细节

3.1 场景创建

基于 `simple_car` 任务，修改了 `mjcf` 描述文件：

- **车身定义**：在 XML 中定义了名为 `chassis` 的几何体（Geom），用于碰撞检测的目标索引。
 - **视角调整**：默认摄像机设置为追踪模式，确保主视角始终跟随车辆。
- screenshots/02_scene_loaded.png

3.2 数据获取

在 `simulate.cc` 中通过指针直接访问物理内存：

```
// 获取速度数据
double vx = this->d->qvel[0];
double vy = this->d->qvel[1];
double speed_kmh = std::sqrt(vx*vx + vy*vy) * 3.6;
// 获取加速度用于判断驾驶模式
```

```
double ax = this->d->qacc[0];  
double ay = this->d->qacc[1];
```

数据验证：通过在控制台打印原始数据与转换后的 km/h 数据进行比对，确保了数值的物理意义正确。

3.3 仪表盘渲染

3.3.1 速度表与综合面板

- **实现思路：**将速度、转速、档位等信息整合为一个多行字符串，使用 `snprintf` 进行格式化，并通过左下角 Overlay 显示。
- **代码片段：**

```
char dashboard_text[512];  
std::snprintf(dashboard_text, sizeof(dashboard_text),  
    "ULTIMATE DASHBOARD\n"  
    "SPEED : %-5.1f km/h\n"  
    "GEAR  : [ %c ]",  
    speed_kmh, gear_char);  
mjr_overlay(mjFONT_BIG, mjGRID_BOTTOMLEFT, viewport, dashboard_text, ...);
```

- **效果展示：** `screenshots/05_full_dashboard.png`

3.4 进阶功能

1. **画中画小地图 (Mini-map)：**创建了一个独立的 `mjvCamera`，设置其 `elevation = -90` (垂直向下) 和 `trackbodyid` 为车辆 ID。先在右下角绘制深色背景框，再调用 `mjr_render` 将该摄像机的视口内容叠加在主画面之上。
2. **驾驶模式与油耗：**实现了自适应驾驶模式。当监测到加速度 `acc_val > 2.0` 或高速行驶时，自动切换至 "SPORT" 模式，此时油耗计算倍率增加 4 倍，模拟了真实车辆的燃油特性。
3. **碰撞检测系统：**利用 `mj_name2id` 获取车身几何体 ID，每帧遍历 `d->contact` 数组。一旦检测到车身与环境发生接触，立即将 `is_crashed` 标志位置位，并触发屏幕中央的红色警告框。

- **实现思路：**编写了一个基于速度阈值的状态机。
- **代码片段：**

```
if (speed_kmh > 1.0) {  
    if (speed_kmh < 15) gear_char = '1';  
    else if (speed_kmh < 30) gear_char = '2';  
    // ... 更多档位  
} else if (speed_kmh < -1.0) gear_char = 'R';  
else gear_char = 'P';
```

四、遇到的问题 and 解决方案

问题1: mjGRID_CENTER 编译报错

- **现象:** 在尝试将警告文字居中显示时，使用了 `mjGRID_CENTER`，导致编译器报错 `'mjGRID_CENTER' was not declared in this scope`。
- **原因:** 查阅 MuJoCo 头文件发现，其 Overlay 系统仅定义了四个角的枚举值（如 `mjGRID_TOPLEFT`），并不支持直接的 Center 枚举。
- **解决:** 采用了一种巧妙的替代方案：定义一个位于屏幕中央的矩形区域 `alert_rect`，然后使用 `mjGRID_TOPLEFT` 将文字绘制在这个矩形的左上角，从而在视觉上实现了居中效果。

问题2: OpenGL 链接错误

- **现象:** 在初期尝试使用原生 OpenGL (`glBegin`, `glVertex2f`) 进行绘制时，链接器报错 `undefined reference to 'glBegin'`。
- **原因:** `CMakeLists.txt` 中未正确链接系统的 OpenGL 库 (`GL`)，且不同操作系统下的库名称存在差异（Windows/Linux）。
- **解决:** 放弃了复杂的 OpenGL 直接调用，转而全面使用 MuJoCo 封装好的 `mjr_` 系列图形接口。这不仅解决了链接错误，还保证了 UI 风格与仿真环境的统一性，且代码更加简洁。

五、测试与结果

5.1 功能测试

- **档位切换：**控制车辆缓慢加速，观察到档位显示从 P -> 1 -> 2 -> 3 依次变化，逻辑正确。
- **碰撞反馈：**驾驶车辆撞向墙壁，屏幕中央准确弹出了红色的 "COLLISION DETECTED!" 提示。
- **模式切换：**猛踩油门（键盘控制加速），观察到模式从 ECO 变为 SPORT，且油量条下降速度明显加快。

5.2 性能测试

- **帧率：**在 Ubuntu 22.04 虚拟机环境下，即使开启了额外的小地图摄像机渲染，仿真帧率依然保持在 60 FPS 以上，说明 `mjr_` 接口的开销极低。
- **资源占用：**内存占用稳定，未出现泄漏。

5.3 效果展示

- **截图：**(请在此处附上包含小地图和红色警告框的运行截图)
- **视频链接：**(请在此处填入你的演示视频链接)

六、总结与展望

6.1 学习收获

通过本次作业，我深入理解了 MuJoCo 物理引擎的数据结构设计，学会了如何在大型 C++ 工程中定位核心逻辑并进行功能扩展。特别是“小地图”和“碰撞检测”的实现，让我对计算机图形学中的坐标变换和物理仿真中的接触动力学有了直观的认识。

6.2 不足之处

目前的仪表盘主要由文字和简单的几何矩形构成，虽然功能完备，但在视觉美观度上不如纹理贴图（Texture Mapping）方案精致。指针式仪表的动画效果也主要通过数字模拟，缺乏真实机械仪表的阻尼感。

6.3 未来改进方向

未来可以尝试引入 `stb_image` 等库加载真实的 PNG 仪表盘素材，利用 OpenGL 着色器实现更炫酷的动态效果。同时，可以结合 MPC 控制算法，将仪表盘不仅作为显示工具，更作为调试控制参数的可视化终端。

七、参考资料

[1] MuJoCo Official Documentation: <https://mujoco.org/> [2] MuJoCo MPC Repository: https://github.com/google-deepmind/mujoco_mpc [3] C++ 课程大作业说明书 (大作业.pdf)